

Part II — Adaptive planning in `cmbagent_lg`

Multi-Agent Systems and Agentic AI — Part II written individually by Zixuan Wang (zw499).

Studied in full at commit d7d0592 (adaptive-planning branch), module `cmbagent_lg/adaptive_planning` (graph, nodes, state, schemas, prompts), which composes the `planning/` and `deep_research/` modules.

II.1 Workflow diagram

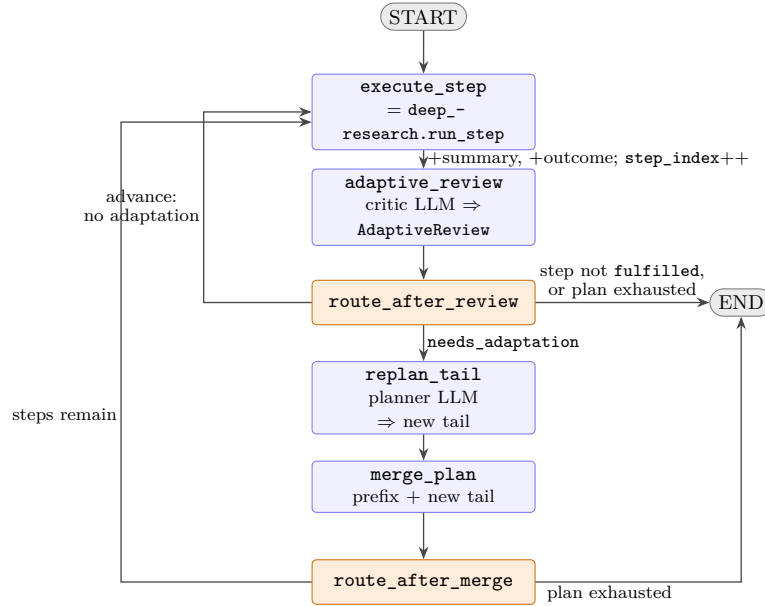


Figure 1: Adaptive-planning LangGraph graph (`adaptive_planning/graph.py`). Blue = work nodes, orange = conditional routers; the threaded `AdaptivePlanState` carries `plan`, `step_index`, `step_summaries/step_outcomes`, `adaptive_review`, `new_tail`, `adaptive_history`. (Excluded from the 2-page limit.)

Walkthrough. The initial `Plan` comes from the bounded propose–critique loop in `planning/`; this graph executes it. `execute_step` (`deep_research`’s `run_step`, reused verbatim) runs the step at `step_index`, appends a summary and a structured outcome `{fulfilled,...}`, increments `step_index`, and always passes to `adaptive_review`. *Trigger*: `adaptive_review` — a critic LLM given the overall task, the step’s summary+outcome, and the unrun steps — returns `AdaptiveReview(needs_adaptation, recommendations)`; `route_after_review` goes to `replan_tail` iff the step was fulfilled, the plan is not exhausted, and `needs_adaptation` — otherwise it advances or halts at `END`. *Mutable vs. fixed*: only the `tail plan.sub_tasks[n_done:]` changes — `replan_tail` gets a replacement tail from the planner LLM (given the completed summaries, current tail, and recommendations) and `merge_plan` sets `plan = completed_prefix + new_tail`. The completed prefix, its summaries/outcomes, and `step_index` are frozen, so executed work is never re-run. *Termination*: a router returns `END` when a step is *not* fulfilled or `step_index > len(plan)`. `execute_step` advances `step_index` by one per step and revision never does, so the loop ends only once the replanner returns a consumable tail — a *soft* guarantee: it is merely *prompted* to “keep the tail minimal”, with no revision counter or length cap, so a non-shrinking replanner is stopped only by LangGraph’s default `recursion_limit` (which *raises* rather than finishing), unlike the initial loop’s hard `num_rounds` bound.

II.2 Problems

P1 — No designed termination or progress guarantee. `step_index` increases only in `execute_step`; `replan_tail` may return a tail *longer* than the steps it replaces, and `merge_plan` routes straight back into execution. There is no revision budget, no plan-length cap, and no monotone quantity that must strictly decrease. Closed-loop control is only well-behaved when the feedback law contracts toward the goal (a termination/Lyapunov argument); a free-text “keep it minimal” instruction to an LLM is not such a contraction, and can thrash or diverge. The only hard backstop, `recursion_limit`, surfaces as an *exception*, not a finished run — and the safeguard the initial planner relies on (`num_rounds`) was dropped here.

P2 — Adaptation is disabled exactly when it is most needed. `route_after_review` returns END the instant an outcome is `fulfilled=False` — and it checks this *before* `needs_adaptation`, so the `adaptive_review` LLM still runs and its verdict is silently discarded. The loop therefore replans after *successful* steps but *gives up* on a failed one, with no repair, retry, or route-around. This inverts the purpose of closed-loop control, which exists to reject disturbances and correct errors: here good news triggers feedback while bad news triggers a halt.

P3 — The replanned tail bypasses the reviewer that gates the initial plan. `planning/` accepts a plan only after a bounded `planner↔plan_reviewer` propose–critique loop. The adaptive replan has *no* reviewer: `merge_plan` splices the planner’s `new_tail` unchecked. Revisions — generated mid-run under the pressure of a surprising result, i.e. exactly when error is likeliest — thus receive *weaker* quality control than the original plan, and can drift from `main_task` or contradict the frozen prefix with nothing to catch it.

P4 — Myopic feedback at unconditional cost. `adaptive_review` sees only the *last* step’s summary+outcome and the remaining steps — not the full trajectory or the artifacts produced — so its decision is greedy (Markov on the last step) and cross-step drift is invisible; closed-loop control needs full-state feedback, not a single observation. It also fires after *every* step regardless of need (and the planner on every revision), so token cost and latency scale with plan length with no gating.

II.3 Proposed improvements

P1 → bound the loop. Thread a `revision_budget` (and/or `max_steps`) through `AdaptivePlanState`, decrement it in `merge_plan`, and have the routers *finish gracefully* (run out the current tail, then END) once it hits zero; optionally reject any `new_tail` that would push the plan past a length cap. *Why*: reinstates a strictly-decreasing quantity, so termination no longer rests on LLM goodwill. *Trade-off*: a hard cap can truncate a genuinely expanding task — so surface “budget exhausted” as an explicit, inspectable outcome rather than a silent stop.

P2 → repair on failure instead of halting. On `fulfilled=False`, route to `replan_tail` (or a dedicated `repair` node) with a per-step retry budget, halting only after K failed attempts. *Why*: makes the loop genuinely closed over errors — the failure becomes the feedback that drives a re-plan, which is the entire point of adaptive planning. *Trade-off*: extra execution/LLM cost and a risk of looping on an impossible step, both bounded by the retry budget; needs a recoverable-vs-fatal flag in the outcome.

P3 → validate the replanned tail. Before `merge_plan`, pass `new_tail` through the existing `planning plan_reviewer` for one critique round against `main_task` and the frozen prefix, regenerating if it diverges. *Why*: gives revisions the same quality gate as the initial plan, closing the asymmetry in P3. *Trade-off*: one extra reviewer call (and possibly a planner re-pass) per revision — bound it to a single round to cap added latency.

P4 → richer, gated feedback. Give the reviewer the full step history and original goal (global state), and *gate* the call — use a cheap model (or skip it) when the outcome shows no downstream impact, escalating to the full critic only on flagged steps. *Why*: full-state feedback catches multi-step drift; gating removes the per-step tax. *Trade-off*: more context costs tokens, and gating risks missing a non-obvious adaptation — so gate conservatively.